



UNIVERSIDAD NACIONAL
EXPERIMENTAL DE GUAYANA

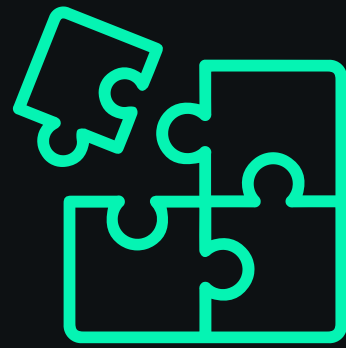
Lenguajes de Programación

Paradigmas, propósito,
estructuras morfológicas y
sintácticas

Presentado por:

Giovanni Martinez
Mansour Ortega
Keiner Frontado
Edgar Morales

UN INGENIERO FRENTE AL SOFTWARE Y EL LENGUAJE



Solución a una problemática

Tras definir la solución de software para un problema, la primera decisión crítica de un ingeniero en informática es la selección del lenguaje de programación.



Criterio Basado en el Enfoque

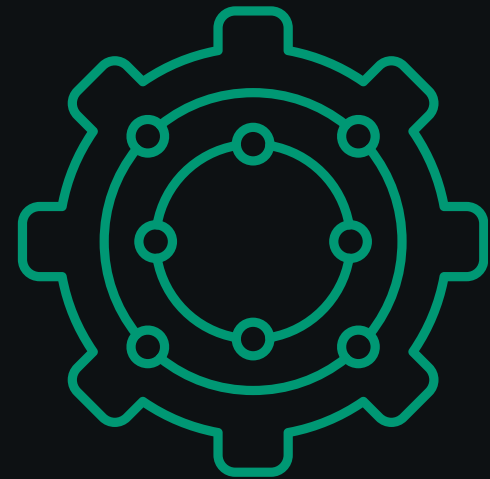
La elección no es al azar; depende estrictamente de las necesidades del proyecto (Orientación a Objetos, Gestión de Bases de Datos, Entornos Web o Sistemas Distribuidos).



Infraestructura Interna

Cada lenguaje posee una identidad única determinada por su morfología (procesamiento de tokens) y su mecanismo de traducción computacional

CONCEPTOS FUNDAMENTALES



Abstracción Lingüística

En la ciencia de la computación, representa el nivel de distanciamiento que existe entre la sintaxis de un lenguaje de programación y las instrucciones de código máquina binario que ejecuta directamente el procesador.



Sintaxis Estructurada

la sintaxis define las reglas gramaticales abstractas que gobiernan el orden jerárquico y la combinación de los tokens para que una instrucción o sentencia sea considerada válida por el compilador.



Morfología Léxica

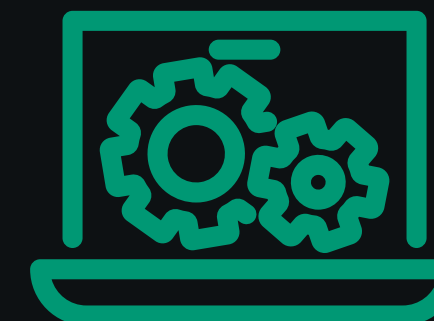
En el área de la teoría de lenguajes de programación que se encarga de estudiar las unidades mínimas con significado propio dentro de un código fuente, conocidas formalmente como tokens.



Traducción Computacional

Es el proceso de transformación de un programa escrito en un lenguaje de alto nivel (código fuente) a un formato ejecutable por la máquina.

Compilado



Interpretado



PARADIGMAS DE LA PROGRAMACIÓN

Entendemos un paradigma como el modelo o patron para entender algo, guiar una investigación o resolver un problema. En este sentido, podemos inferir que en la programación toma especial importancia esta terminología.

La programación evoluciona constantemente, de igual manera sus paradigmas.

Como existen los paradigmas consolidados, existen los paradigmas emergentes y de igual manera, la filosofía personal de programación puede ser considerada un paradigma



PARADIGMAS A TRAVÉS DEL TIEMPO

Distinción cronológica de los paradigmas

Para facilitar la comprensión del tema, podemos separar los paradigmas de manera cronológica en 2 bloques

B1

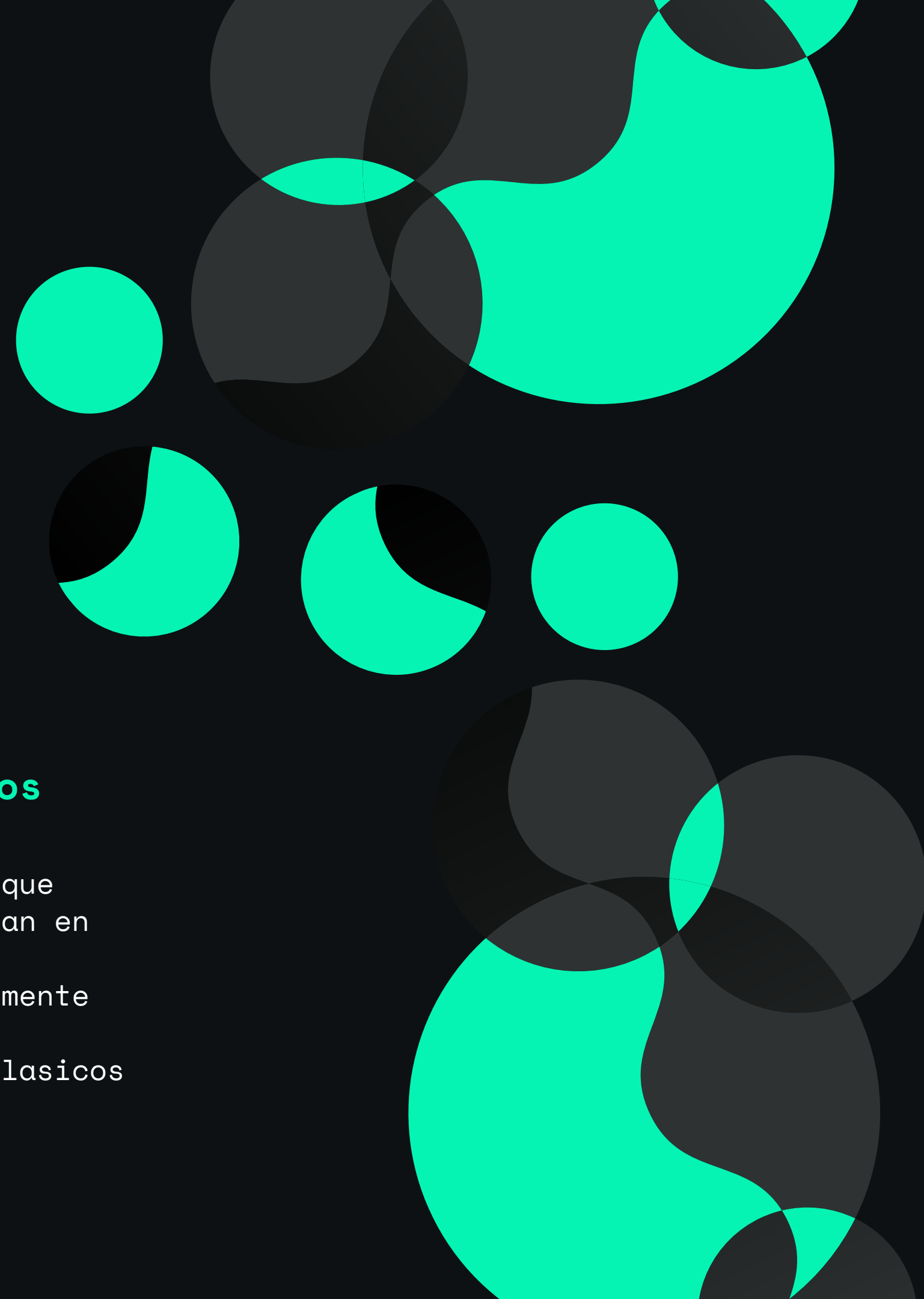
Paradigmas Clásicos

Nos referimos a los paradigmas clásicos como aquellos que son los primeros no solo cronológicamente sino también aquellos que sentaron las bases y fundamentos de la programación

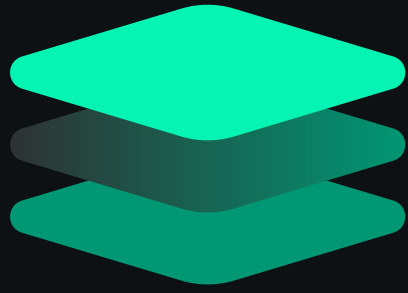
B2

Paradigmas Modernos

Con paradigmas modernos hablamos de paradigmas que emergen y/o se consolidan en la actualidad, son desarrollados principalmente con el fin de solventar problemáticas que los clásicos no pueden



PARADIGMAS DE LA PROGRAMACIÓN



Bloque 1 y 2.

Paradigmas de bloque 1:
Estructural y orientado a
objetos

Paradigmas de bloque 2:
Funcional, declarativo y
concurrente.



Paradigma Estructural.

Consiste en dar instrucciones
directas, explícitas y
secuenciales a la máquina, con
las cuales se le indica como
debe cambiar su estado.



Paradigma Orientado a Objetos.

Este enfoque permite modelar el
software a partir de conceptos del
mundo real mediante “objetos”
solucionando diversos problemas en
el proceso.



Paradigma Funcional.

Este paradigma trata a la
computación como la evaluación
de matemáticas puras y evita
los datos mutables.



Paradigma Declarativo.

Basado en declarar que
resultado se desea obtener
analizando de forma aislada el
proceso de cómo la computadora
lo calcula.



Paradigma Concurrente.

El paradigma concurre se basa
en descomponer un programa en
unidades de ejecución
independientes para así
mejorar el rendimiento.

MATRIZ DESCRIPTIVA DE PARADIGMAS

Paradigma	Gestión del Estado y Secuenciación	Mutabilidad y Memoria	Abstracción y Características	Mitigación de Efectos secundarios / control
Imperativo	Explícita e instruccional.	Alta mutabilidad.	Procedimental.	Ninguna.
Orientado a objetos	Dirigida por mensajes.	Mutabilidad controlada.	Modelado del dominio.	Aislamiento por visibilidad.
Funcional	Evaluación de expresiones.	Inmutabilidad estricta.	Funciones de primer orden.	Eliminación programática.
Declarativo	Ingerencia automática.	Gestión implícita.	Cláusulas de Horn.	Abstracción total.
Concurrente	Asíncrona por el paso de mensajes.	Confinamiento estricto.	Aislamiento de entidades.	Mitigación lingüística.

JUSTIFICACIÓN CONCEPTUAL DEL LENGUAJE L

01

Dominio Crítico ECO-GRID

Se plantea como solución un Lenguaje de Dominio Específico (DSL) diseñado explícitamente para el resguardo y balance autónomo de microrredes de energía, suprimiendo las abstracciones redundantes de los entornos de propósito general.

02

Enfoque Imperativo y Estructurado

Se implementa una sintaxis lineal y determinista que permite la fácil interpretación por parte de operadores de planta, garantizando la predictibilidad absoluta de la ejecución física sobre sensores, inversores y relés de alta potencia.

ANALIZADOR LÉXICO: FRONTERA DE TOKENS

Identificadores

ID

Cadenas destinadas a nombrar variables y hardware de la planta. Deben comenzar estrictamente con letra minúscula y aceptar guiones bajos.

Patrón: `[a-z][a-z0-9_]*`

Literales Numéricos

NUM

Constantes de precisión que representan mediciones de caudal de carga, voltajes y temperaturas. Admite valores enteros y de punto flotante.

Ej: 55, 90, 55.5

Estados Físicos

ESTADO

Literales lógicos simplificados para el operario que actúan directamente sobre actuadores físicos. Evita el uso de tipos booleanos genéricos.

Valores: ON | OFF

Fin de Instrucción

Se descarta formalmente el punto y coma tradicional. El salto de línea (`\n`) actúa como el delimitador unívoco para simplificar el código.

Sintaxis limpia de bloques

ANALIZADOR SINTÁCTICO Y REGLAS BNF

Reglas de Producción

1. **Punto de Entrada:** Obliga a que todo script inicie estrictamente con el comando de calibración `init_grid`.
2. **Clausura Estricta:** Los bloques de toma de decisiones e iteraciones deben cerrarse de forma explícita con las directivas `fin_si` o `fin_mientras`.
3. **Unificación Termino:** La gramática agrupa variables, literales y llamadas a funciones bajo la regla común `<Termino>` para evitar ambigüedades.

```
<Programa> ::= "init_grid" "\n" <ListaSentencias>
<Sentencia> ::= <Asignacion> "\n"
               | <LlamadaFuncion> "\n"
               | <SentenciaCondicional> "\n"
               | <SentenciaIterativa> "\n"

<SentenciaCondicional> ::= "si" <Expresion> "entonces" "\n"
<ListaSentencias> "fin_si"
<SentenciaIterativa> ::= "mientras" <Expresion> "ejecutar" "\n"
<ListaSentencias> "fin_mientras"

<LlamadaFuncion> ::= "leer_temperatura" "(" ID ")"
                  | "estado_carga" "(" ID ")"
                  | "conmutar_linea" "(" ID "," ESTADO ")"
                  | "emitir_alerta" "(" STRING ")"

<Expresion> ::= <Termino> OP_REL <Termino> | BOOL
```

PREVENCIÓN DE FUGA TÉRMICA EN BATERÍAS

La fuga térmica es una reacción química exotérmica destructiva e irreversible en las celdas de litio. Se presenta el diseño lógico-físico en tres fases consecutivas para neutralizar el desbordamiento.



Vigilancia Crítica

Se define un monitoreo ininterrumpido en tiempo real sobre los sensores del banco `bateria_litio_1`, estableciendo un límite de alerta térmica infranqueable en los 55°C.



Alivio e Inyección

Al registrarse el desbordamiento de temperatura, el sistema suspende inmediatamente la carga solar de entrada e independiza las cargas del sector industrial aislando su conmutador de alimentación.



Respaldo y Alarma

Se canaliza de forma automática el suministro eléctrico desde la red comercial externa para conservar la operatividad de la planta mientras se disparan alarmas visuales constantes en la consola del HMI.

CASO DE ESTUDIO A

CÓDIGO OPERATIVO EN LENGUAJE L

1. Monitoreo Infinito

La iteración evalúa la constante lógica de verdad garantizando el muestreo del hardware de forma constante e indefinida en la planta.

2. Captura y Asignación

La lectura del termómetro analógico es guardada sincronamente mediante el token de asignación directa =.

3. Bifurcación Condicional

Un flujo de conmutación de estado física aísla y transfiere la carga, protegiendo de forma completa el componente químico en peligro.



```
// Inicialización del sistema ECO-GRID
init_grid

mientras verdadero ejecutar
    temp_bateria = leer_temperatura(bateria_litio_1)
    si temp_bateria > 55 entonces
        conmutar_linea(sistema_refrigeracion, ON)
        conmutar_linea(paneles_solares_entrada, OFF)
        conmutar_linea(sector_industrial, OFF)
        conmutar_linea(red_comercial_respaldo, ON)
        emitir_alerta("ALERTA: Fuga térmica crítica
detectada.")
    fin_si
fin_mientras
```

PROCESAMIENTO EN LAS FASES DEL COMPILADOR

01

Fase Léxica

El escáner analiza carácter por carácter, suprimiendo comentarios e ignorando espacios, dividiendo el código en lexemas como `ID`, `OP_REL` y `ESTADO`.

02

Fase Sintáctica

El parser valida la estructura estructural comparando los tokens contra la gramática BNF para armar secuencialmente el Árbol de Sintaxis Abstracta (`AST`) de la ejecución.

03

Semántica

Se verifica la correspondencia lógica de tipos, asegurando que las comparaciones lógicas sean entre datos del mismo tipo, y que el relé admita únicamente `ON` u `OFF`.

04

Generación

El compilador conceptual traduce las instrucciones validadas del DSL en señales a nivel de controlador para accionar directamente los actuadores analógicos y disparar alarmas visuales.

CASO DE ESTUDIO B

BALANCE DE CARGA AUTÓNOMO (OPTIMIZACIÓN ENERGÉTICA)

Para el segundo escenario, el enfoque del código cambia hacia la eficiencia y la toma de decisiones autónomas según el nivel de almacenamiento. Este programa realiza una evaluación secuencial punto a punto-



Por encima de 90%

Si las baterías están casi llenas, el sistema determina que hay superávit energético y acciona automáticamente un relé para inyectar y vender ese excedente a la red pública general.



Por debajo de 20%

Se ejecuta una desconexión selectiva de cargas. Aplica un criterio de prioridad donde apaga por completo las áreas no esenciales para racionar y garantizar el suministro eléctrico ininterrumpido en las zonas de alta prioridad, que son el sector médico y los servidores críticos.



CASO DE ESTUDIO B

CÓDIGO OPERATIVO EN LENGUAJE L



```
init_grid
```

```
carga_actual = estado_carga(bateria_respaldo_c2)
```

```
si carga_actual > 90 entonces
```

```
    conmutar_linea(rele_inyeccion_publica, ON)
```

```
    emitir_alerta("INFO: Baterías óptimas. Inyectando excedente a la red pública.")
```

```
fin_si
```

```
si carga_actual < 20 entonces
```

```
    conmutar_linea(sector_fabrica_no_esencial, OFF)
```

```
    conmutar_linea(sector_servidores_criticos, ON)
```

```
    conmutar_linea(sector_medico, ON)
```

```
    emitir_alerta("ALERTA: Carga crítica en horas pico. Sectores no esenciales aislados.")
```

```
fin_si
```




CONCLUSIONES

Comprender la infraestructura interna de un lenguaje transforma al informático de un simple usuario de herramientas a un arquitecto de software de alto nivel.

El benchmarking demuestra que no hay lenguajes mejores que otros en términos absolutos; **la clave** radica en seleccionar el mecanismo de ejecución adecuado para el hardware y la latencia del proyecto.

El diseño del Lenguaje L evidencia que un DSL con una **morfología y sintaxis** (BNF) bien estructuradas **elimina la ambigüedad**, permitiendo un control autónomo y seguro en entornos críticos como ECO-GRID.

**¡ MUCHAS
GRACAS POR SU
ATENCIÓN !**

